

Laporan Akhir Tugas Besar
Pemrograman Berorientasi Objek
PERANCANGAN APLIKASI PENCARIAN RUTE
TERCEPAT BERDASARKAN PETA TELKOM
UNIVERSITY BERBASIS MOBILE



Dibuat oleh:

Putri Rahayu Damayanti	103012400277
Hamam Nashiruddin	103012400118
Najla Tsabita Afiyah	103012400305
Putri Ayu Lestari	103012430055

S1 Informatika
Fakultas Informatika
Universitas Telkom
2026

DAFTAR ISI

1.1 Latar Belakang.....	6
1.2 Tujuan.....	6
1.3 Deskripsi Umum Sistem (Ruang Lingkup).....	6
1.4 Metodologi Pengembangan Sistem.....	7
1.4.1 Analisis Kebutuhan.....	7
1.4.2 Perancangan Sistem.....	7
1.4.3 Implementasi.....	7
1.4.4 Pengujian.....	8
2.1 Pemrograman Berorientasi Objek.....	8
2.2 Unified Modeling Language (UML).....	9
2.3 Java.....	9
2.4 Springboot.....	9
2.5 React Native.....	9
2.6 PostgreSQL.....	10
2.7 Algoritma A* (A-Star Search).....	10
3.1 Analisis Kebutuhan Sistem.....	10
3.1.1 Kebutuhan Fungsional (Fitur).....	10
3.1.1.1. Fitur Autentikasi.....	10
3.1.1.2. Fitur Pemilihan Titik Asal dan Tujuan.....	11
3.1.1.3. Fitur Pemilihan Moda Transportasi.....	11
3.1.1.4. Fitur Pencarian Rute Terpendek.....	11
3.1.1.5. Fitur Kalkulasi Jarak dan Estimasi Waktu.....	11
3.1.1.6. Fitur Tampilan Hasil Rute.....	11
3.1.1.7. Fitur Riwayat Perjalanan.....	11
3.1.1.8. Fitur Pengaturan Akun dan Tema.....	11
3.1.2 Kebutuhan Non-Fungsional.....	12
3.1.2.1. Spesifikasi Perangkat Lunak.....	12
3.1.2.2. Spesifikasi Perangkat Keras.....	12
3.2 Implementasi Basis Data.....	12
3.2.1 Entity Relationship Diagram (ERD).....	12
3.2.2 Struktur Basis data.....	15
3.2.2.1. Tabel User.....	15
3.2.2.2. Tabel History.....	15
3.2.2.3. Tabel MainNode.....	16
3.2.2.4. Tabel IntersectionNode.....	16
3.2.2.5. Tabel MainEdge.....	16
3.2.2.6. Tabel IntersectionEdge.....	17
3.3 Implementasi UML.....	18
3.3.1 Class Diagram.....	18
3.3.2 Deskripsi Kelas.....	18
3.3.2.1. Kelas User.....	18
3.3.2.2. Kelas Node (Abstract).....	19
3.3.2.3. Kelas MainNode.....	19

3.3.2.4. Kelas IntersectionNode.....	19
3.3.2.5. Kelas Edge (Abstract).....	19
3.3.2.6. Kelas MainEdge.....	20
3.3.2.7. Kelas IntersectionEdge.....	20
3.3.2.8. Kelas AStarPathfinder.....	20
3.3.2.9. Kelas History.....	21
3.3.2.10. Kelas HistoryManager.....	21
3.3.2.11. Kelas RouteResult.....	21
3.4 Implementasi Antarmuka Pengguna (User Interface).....	21
3.4.1 Halaman Login.....	21
3.4.2 Halaman Registrasi.....	22
3.4.3 Halaman Forgot Password.....	22
3.4.4 Halaman Utama (Main Page).....	22
3.4.5 Halaman Pengaturan (Setting Page).....	22
4.1 Tujuan Pengujian.....	22
4.2 Lingkungan Pengujian.....	23
4.2.1 Perangkat Lunak Pengujian.....	23
4.2.2 Perangkat Keras Pengujian.....	23
4.3 Metode Pengujian.....	23
4.3.1 (Black Box/White Box) Testing.....	23
4.3.2 Skenario Pengujian.....	24
5.1 Fitur Autentikasi.....	25
5.1.1 Kasus uji.....	25
5.1.2 Hasil.....	25
5.2 Fitur Pemilihan Titik Asal dan Tujuan.....	26
5.2.1 Kasus uji.....	26
5.2.2 Hasil.....	26
5.3 Fitur Pemilihan Moda Transportasi.....	27
5.3.1 Kasus Uji.....	27
5.3.2 Hasil.....	27
5.4 Fitur Pencarian Rute Terpendek.....	28
5.4.1 Kasus uji.....	28
5.4.2 Hasil.....	28
5.5 Fitur Kalkulasi Jarak dan Estimasi Waktu.....	29
5.5.1 Kasus uji.....	29
5.5.2 Hasil.....	29
5.6 Fitur Tampilan Hasil Rute.....	30
5.6.1 Kasus uji.....	30
5.6.2 Hasil.....	30
5.7 Fitur Riwayat Perjalanan.....	30
5.7.1 Kasus uji.....	30
5.7.2 Hasil.....	31
5.8 Fitur Pengaturan Akun dan Tema.....	31
5.8.1 Kasus uji.....	31

5.8.2 Hasil.....	31
6.1 Kesimpulan.....	32
6.2 Saran.....	33
Lampiran A Pembagian Tugas.....	35
Lampiran B Source Code Penting.....	35
Lampiran C Screenshot Sistem.....	35
Lampiran D Manual Instalasi Sistem.....	35

DAFTAR TABEL

Tabel 1 Tabel Entitas User.....	18
Tabel 2 Tabel Entitas History.....	19
Tabel 3 Tabel Entitas MainNode.....	19
Tabel 4 Tabel Entitas IntersectionNode.....	20
Tabel 5 Tabel Entitas MainEdge.....	20
Tabel 6 Tabel Entitas IntersectionEdge.....	20
Tabel 7 Identifikasi Pengujian.....	27
Tabel 8 Kasus Uji Fitur Autentikasi.....	28
Tabel 9 Hasil Uji Fitur Autentikasi.....	29
Tabel 10 Hasil Uji Fitur Pemilihan Titik Asal dan Tujuan.....	29
Tabel 11 Hasil Uji Fitur Pemilihan Titik Asal dan Tujuan.....	30
Tabel 12 Hasil Uji Fitur Pemilihan Moda Transportasi.....	30
Tabel 13 Hasil Uji Fitur Pemilihan Moda Transportasi.....	30
Tabel 14 Hasil Uji Fitur Pencarian Rute Terpendek.....	32
Tabel 15 Hasil Uji Fitur Pencarian Rute Terpendek.....	32
Tabel 16 Hasil Uji Fitur Kalkulasi Jarak dan Estimasi Waktu.....	32
Tabel 17 Hasil Uji Fitur Kalkulasi Jarak dan Estimasi Waktu.....	33
Tabel 18 Hasil Uji Fitur Tampilan Hasil Rute.....	33
Tabel 19 Hasil Uji Fitur Tampilan Hasil Rute.....	34
Tabel 20 Hasil Uji Fitur Riwayat Perjalanan.....	34
Tabel 21 Hasil Uji Fitur Riwayat Perjalanan.....	34
Tabel 22 Hasil Uji Fitur Pengaturan Akun dan Tema.....	35
Tabel 23 Hasil Uji Fitur Pengaturan Akun dan Tema.....	35

DAFTAR GAMBAR

Gambar 1 ERD.....	15
Gambar 2 Class Diagram UML.....	20
Gambar 3&4.....	37
Gambar 5&6.....	38
Gambar 7&8.....	39
Gambar 9&10.....	39
Gambar 11.....	40

1 Pendahuluan

1.1 Latar Belakang

Telkom University memiliki area kampus yang cukup luas dengan banyak gedung dan fasilitas di dalamnya, sehingga mahasiswa, dosen, staf, maupun tamu kampus sering mengalami kesulitan dalam menemukan rute tercepat menuju lokasi yang diinginkan. Kondisi ini dapat menghambat mobilitas dan efisiensi waktu pengguna, terutama bagi mereka yang baru pertama kali mengunjungi kawasan kampus.

Untuk mengatasi permasalahan tersebut, diperlukan sebuah sistem navigasi yang dirancang khusus untuk lingkungan kampus Telkom University. Sistem ini diharapkan mampu membantu pengguna menentukan jalur tercepat secara efisien dan akurat berdasarkan peta kawasan kampus. Dengan memanfaatkan algoritma pencarian rute A* (*A-Star*) dan data peta kampus dalam format GeoJSON, aplikasi ini dapat menghitung rute optimal dengan mempertimbangkan aksesibilitas tiga moda transportasi, yaitu pejalan kaki, sepeda motor, dan mobil.

1.2 Tujuan

Tujuan dari pembangunan sistem ini adalah merancang dan mengembangkan aplikasi mobile pencarian rute tercepat berbasis peta Telkom University yang dapat membantu pengguna menentukan jalur optimal di area kampus secara efisien dan akurat. Secara lebih rinci, tujuan sistem ini adalah:

1. Mempermudah pengguna dalam menemukan rute menuju gedung atau fasilitas di lingkungan kampus Telkom University.
2. Mengoptimalkan waktu tempuh dengan memberikan rekomendasi jalur tercepat berdasarkan algoritma A* menggunakan heuristik haversine.
3. Mendukung tiga moda transportasi (pejalan kaki, sepeda motor, dan mobil) dengan mempertimbangkan aksesibilitas jalan untuk masing-masing moda.
4. Menampilkan informasi perjalanan berupa estimasi jarak dalam meter dan durasi tempuh berdasarkan kecepatan rata-rata tiap moda transportasi.
5. Menyediakan antarmuka mobile yang intuitif dan responsif yang dapat diakses di platform iOS maupun Android.

1.3 Deskripsi Umum Sistem (Ruang Lingkup)

Sistem yang dibangun adalah aplikasi mobile pencarian rute tercepat berbasis peta Telkom University. Aplikasi ini dirancang khusus untuk memetakan rute di dalam kawasan internal kampus Telkom University Bandung dan tidak

mencakup area di luar kawasan kampus. Pengguna dapat melakukan autentikasi menggunakan akun Google melalui mekanisme OAuth, kemudian menentukan titik asal dan tujuan secara interaktif langsung di peta kampus. Sistem mendukung tiga moda transportasi yaitu pejalan kaki, sepeda motor, dan mobil, di mana setiap moda memiliki aksesibilitas jalan yang berbeda-beda. Rute optimal dihitung menggunakan algoritma A* dengan heuristik haversine dan ditampilkan sebagai polyline di atas peta, disertai informasi estimasi jarak dan durasi perjalanan. Sistem terdiri dari dua komponen utama, yaitu backend REST API berbasis Spring Boot yang menjalankan algoritma pencarian, dan frontend aplikasi mobile berbasis React Native yang berjalan di platform iOS dan Android.

1.4 Metodologi Pengembangan Sistem

Pengembangan sistem ini dilakukan secara bertahap mulai dari analisis kebutuhan hingga pengujian sistem. Berikut adalah tahapan-tahapan yang dilakukan:

1.4.1 Analisis Kebutuhan

Tahap ini dilakukan dengan mengidentifikasi kebutuhan fungsional dan non-fungsional sistem. Kebutuhan fungsional yang diidentifikasi mencakup fitur autentikasi pengguna menggunakan akun Google, tampilan peta kawasan kampus Telkom University, pemilihan titik asal dan tujuan secara interaktif di peta, pemilihan moda transportasi, serta penampilan hasil rute beserta estimasi jarak dan durasi perjalanan. Kebutuhan non-fungsional mencakup spesifikasi perangkat keras dan perangkat lunak yang dibutuhkan baik pada lingkungan pengembangan maupun lingkungan produksi.

1.4.2 Perancangan Sistem

Tahap perancangan dilakukan dengan menyusun rancangan sistem sebelum proses implementasi dimulai. Pada tahap ini dibuat *class diagram* untuk menggambarkan struktur kelas dan hubungan antar kelas pada sistem, serta *mockup* antarmuka pengguna menggunakan Figma yang mencakup halaman login, registrasi, lupa password, halaman utama peta, dan halaman pengaturan. Rancangan ini dijadikan acuan selama proses implementasi berlangsung.

1.4.3 Implementasi

Tahap implementasi dilakukan berdasarkan hasil perancangan yang telah disusun sebelumnya. Implementasi dikerjakan pada dua komponen utama secara paralel. Backend diimplementasikan menggunakan Java 21 dan Spring Boot 3.3.13 dengan Maven sebagai *build tool*, mencakup implementasi algoritma A* dengan heuristik haversine, profil moda transportasi untuk pejalan kaki, sepeda motor, dan mobil, pemrosesan data peta kampus dalam format GeoJSON, serta validasi token JWT dari Supabase Auth. Frontend diimplementasikan menggunakan React Native 0.81.5 via Expo 54

dengan TypeScript, mencakup tampilan peta interaktif berbasis *react-native-maps*, alur pencarian rute, pemilihan moda transportasi, serta pengaturan tema aplikasi.

1.4.4 Pengujian

Tahap pengujian dilakukan untuk memastikan sistem berjalan sesuai dengan kebutuhan yang telah diidentifikasi. Pengujian backend menggunakan Spring Boot Test dengan H2 sebagai database *in-memory* sehingga pengujian dapat berjalan tanpa memerlukan koneksi ke PostgreSQL eksternal. Pengujian frontend menggunakan Jest dan Testing Library React Native, mencakup unit test untuk layanan autentikasi (*auth-service*), layanan GeoJSON (*geojson-service*), layanan pencarian rute (*routing-service*), dan fungsi utilitas koordinat (*coords*).

2 Dasar Teori

Bagian ini membahas teori-teori dasar yang menjadi landasan dalam perancangan dan pengembangan sistem pencarian rute tercepat berbasis peta Telkom University. Teori-teori yang dibahas meliputi konsep pemrograman berorientasi objek, pemodelan sistem menggunakan UML, teknologi dan *framework* yang digunakan pada sisi *backend* maupun *frontend*, basis data, serta algoritma pencarian rute yang menjadi inti dari sistem ini.

2.1 Pemrograman Berorientasi Objek

Pemrograman Berorientasi Objek (*Object-Oriented Programming/OOP*) adalah paradigma pemrograman yang mengorganisasikan perangkat lunak dalam bentuk objek-objek yang saling berinteraksi, di mana setiap objek merupakan instansiasi dari sebuah kelas yang mendefinisikan atribut (data) dan method (perilaku). OOP memiliki empat prinsip utama. Pertama, enkapsulasi (*encapsulation*), yaitu pembungkusan data dan method dalam satu kesatuan kelas serta pembatasan akses terhadap data melalui modifier seperti *private* dan *protected*, sehingga data hanya dapat diubah melalui method yang telah ditentukan (*getter/setter*). Kedua, pewarisan (*inheritance*), yaitu kemampuan suatu kelas untuk mewarisi atribut dan method dari kelas lain, sehingga mengurangi duplikasi kode. Ketiga, polimorfisme (*polymorphism*), yaitu kemampuan suatu method untuk memiliki perilaku berbeda tergantung pada objek yang memanggilnya. Keempat, abstraksi (*abstraction*), yaitu menyembunyikan detail implementasi yang kompleks dan hanya menampilkan fungsionalitas penting kepada pengguna kelas tersebut.

Pada sistem ini, prinsip OOP diterapkan secara konsisten, antara lain melalui kelas abstrak *Node* dan *Edge* yang menjadi induk bagi kelas-kelas turunannya, enkapsulasi data pada seluruh kelas domain (*User*, *History*, *dst.*) dengan atribut *private/protected* dan akses terkontrol melalui *getter/setter*, serta abstraksi pada kelas *AStarPathfinder* yang menyembunyikan kompleksitas algoritma pencarian rute di balik method publik *findShortestPath()*.

2.2 Unified Modeling Language (UML)

Unified Modeling Language (UML) adalah bahasa pemodelan standar yang digunakan untuk merancang, menggambarkan, dan mendokumentasikan struktur serta perilaku suatu sistem perangkat lunak secara visual. UML terdiri dari berbagai jenis diagram, salah satunya adalah *class diagram* yang digunakan untuk menggambarkan struktur kelas dalam sistem, termasuk atribut, method, serta relasi antar kelas seperti pewarisan (*inheritance*), asosiasi, dan agregasi.

Pada proyek ini, *class diagram* digunakan sebagai alat bantu perancangan sebelum proses implementasi, yang menggambarkan kelas-kelas utama sistem seperti User, Node, Edge, AStarPathfinder, History, dan HistoryManager, beserta relasi pewarisan antara Node–MainNode/IntersectionNode dan Edge–MainEdge/IntersectionEdge, serta relasi asosiasi antara User dan History.

2.3 Java

Java adalah bahasa pemrograman berorientasi objek yang bersifat *platform-independent*, artinya kode yang ditulis dapat dijalankan di berbagai sistem operasi melalui *Java Virtual Machine* (JVM). Java mendukung penuh keempat prinsip OOP dan banyak digunakan dalam pengembangan aplikasi *enterprise* maupun *backend* layanan web.

Pada sistem ini, Java digunakan sebagai bahasa pemrograman utama untuk membangun *backend* aplikasi, khususnya pada implementasi struktur data graf kampus (Node, Edge), algoritma pencarian rute (AStarPathfinder), serta logika bisnis lainnya seperti pengelolaan pengguna dan riwayat perjalanan.

2.4 Springboot

Spring Boot adalah *framework* berbasis Java yang dibangun di atas Spring Framework, dirancang untuk menyederhanakan proses konfigurasi dan pengembangan aplikasi. Spring Boot menyediakan berbagai modul siap pakai (*starter*) yang dapat ditambahkan sesuai kebutuhan, seperti modul untuk membangun REST API, validasi input, keamanan, dan akses data.

Pada sistem ini, Spring Boot digunakan sebagai *framework* utama untuk membangun REST API *backend*, yang menjadi jembatan komunikasi antara algoritma pencarian rute dengan aplikasi *frontend*.

2.5 React Native

React Native adalah *framework* pengembangan aplikasi mobile cross-platform yang memungkinkan satu basis kode dapat dijalankan di platform iOS dan Android. React Native menggunakan bahasa JavaScript/TypeScript dan konsep komponen yang dapat digunakan kembali (*reusable components*), sehingga mempercepat proses pengembangan tanpa harus menulis kode native terpisah untuk masing-masing platform.

Pada sistem ini, React Native digunakan untuk membangun aplikasi *frontend* yang menampilkan peta interaktif, formulir autentikasi, serta antarmuka pencarian dan tampilan rute kepada pengguna.

2.6 PostgreSQL

PostgreSQL adalah sistem manajemen basis data relasional (Relational Database Management System/RDBMS) bersifat open-source yang mendukung berbagai fitur lanjutan seperti tipe data kompleks, full-text search, dan ekstensi geospasial. PostgreSQL banyak digunakan dalam pengembangan aplikasi modern karena stabilitas dan skalabilitasnya.

Pada sistem ini, PostgreSQL digunakan sebagai basis data utama untuk menyimpan data pengguna serta riwayat perjalanan, dengan layanan hosting dan autentikasi disediakan oleh Supabase.

2.7 Algoritma A* (A-Star Search)

Algoritma A* adalah algoritma pencarian jalur terpendek pada graf yang menggabungkan pendekatan greedy dan exhaustive search melalui fungsi evaluasi $f(n) = g(n) + h(n)$. Pada fungsi tersebut, $g(n)$ merepresentasikan biaya aktual yang telah ditempuh dari titik awal hingga node n , sedangkan $h(n)$ merepresentasikan estimasi biaya (heuristik) dari node n menuju titik tujuan. Algoritma ini akan selalu mengeksplorasi node dengan nilai $f(n)$ terkecil terlebih dahulu menggunakan struktur data priority queue.

Agar algoritma A* menjamin hasil yang optimal, fungsi heuristik $h(n)$ harus bersifat admissible, yaitu tidak pernah melebihi-lebihkan biaya sebenarnya menuju tujuan. Pada sistem ini, heuristik yang digunakan adalah jarak haversine — jarak garis lurus antara dua koordinat di permukaan bumi yang memperhitungkan kelengkungan bumi. Heuristik ini bersifat admissible karena jarak tempuh melalui jalan-jalan kampus akan selalu lebih besar atau sama dengan jarak garis lurus antara dua titik tersebut.

3 Analisis dan Implementasi Sistem

3.1 Analisis Kebutuhan Sistem

3.1.1 Kebutuhan Fungsional (Fitur)

Berdasarkan hasil analisis terhadap sistem yang dibangun, berikut adalah kebutuhan fungsional atau fitur-fitur yang terdapat pada aplikasi pencarian rute tercepat berbasis peta Telkom University.

3.1.1.1 Fitur Autentikasi

Sistem menyediakan mekanisme autentikasi pengguna melalui dua jalur, yaitu *login* lokal menggunakan email dan password, serta *login* menggunakan akun Google melalui OAuth. Kelas User menyimpan atribut loginProvider untuk membedakan kedua jalur tersebut. Pengguna yang masuk melalui Google tidak dapat mengubah password melalui sistem, karena kredensial dikelola oleh penyedia OAuth.

3.1.1.2 Fitur Pemilihan Titik Asal dan Tujuan

Pengguna menentukan titik asal dan tujuan dengan cara mengetuk langsung pada peta kampus yang ditampilkan. Titik

yang dipilih disimpan sebagai koordinat dan diteruskan ke proses pencarian rute.

3.1.1.3. Fitur Pemilihan Moda Transportasi

Pengguna dapat memilih salah satu dari tiga moda transportasi, yaitu pejalan kaki (WALK), sepeda motor (MOTORCYCLE), atau mobil (CAR). Setiap moda memiliki aturan aksesibilitas jalan yang berbeda, yang divalidasi melalui method `isAccessibleBy(mode)` pada kelas `Edge`.

3.1.1.4. Fitur Pencarian Rute Terpendek

Sistem menjalankan algoritma A^* melalui kelas `AStarPathfinder` untuk menemukan rute terpendek antara titik asal dan tujuan, sesuai dengan moda transportasi yang dipilih. Pencarian rute menggunakan fungsi heuristik *haversine* untuk mengestimasi jarak antar koordinat.

3.1.1.5. Fitur Kalkulasi Jarak dan Estimasi Waktu

Sistem menghitung total jarak tempuh dan estimasi durasi perjalanan berdasarkan kecepatan rata-rata tiap moda transportasi, melalui method `calculateDistance()` dan `getDuration(mode)` pada kelas `Edge`.

3.1.1.6. Fitur Tampilan Hasil Rute

Rute yang ditemukan ditampilkan sebagai garis (*polyline*) di atas peta, disertai ringkasan jarak dan durasi perjalanan pada panel informasi. Pengguna dapat menghapus rute aktif untuk memulai pencarian baru.

3.1.1.7. Fitur Riwayat Perjalanan

Sistem menyimpan setiap pencarian rute sebagai riwayat perjalanan melalui kelas `History`, yang mencatat waktu pencarian, nama titik asal dan tujuan, jarak, durasi, serta moda transportasi yang digunakan. Pengguna dapat menyaring riwayat berdasarkan kategori waktu (hari ini, kemarin, minggu lalu, bulan lalu) melalui kelas `HistoryManager`.

3.1.1.8. Fitur Pengaturan Akun dan Tema

Pengguna dapat mengubah *username*, password (khusus pengguna LOCAL), dan foto profil melalui halaman pengaturan. Sistem juga mendukung perubahan tema tampilan antara mode terang dan gelap.

3.1.2 Kebutuhan Non-Fungsional

3.1.2.1. Spesifikasi Perangkat Lunak

Saat aplikasi resmi diluncurkan (production), maka dibutuhkan server atau cloud agar pengguna dapat mengaksesnya secara online. Untuk memastikan ketersediaan layanan dan memenuhi beban pemrosesan algoritma pencarian rute di backend secara

real-time, spesifikasi server minimum yang direkomendasikan adalah sebagai berikut:

Processor : Minimal 2 Core vCPU
RAM : 8 Gb
Storage : 128 Gb (SSD)
Internet : Bandwith > 100 Mbps dengan Static IP Public
Sistem Operasi: Ubuntu Server 22.04 atau Linux Server setara yang ringan dan stabil

3.1.2.2. Spesifikasi Perangkat Keras

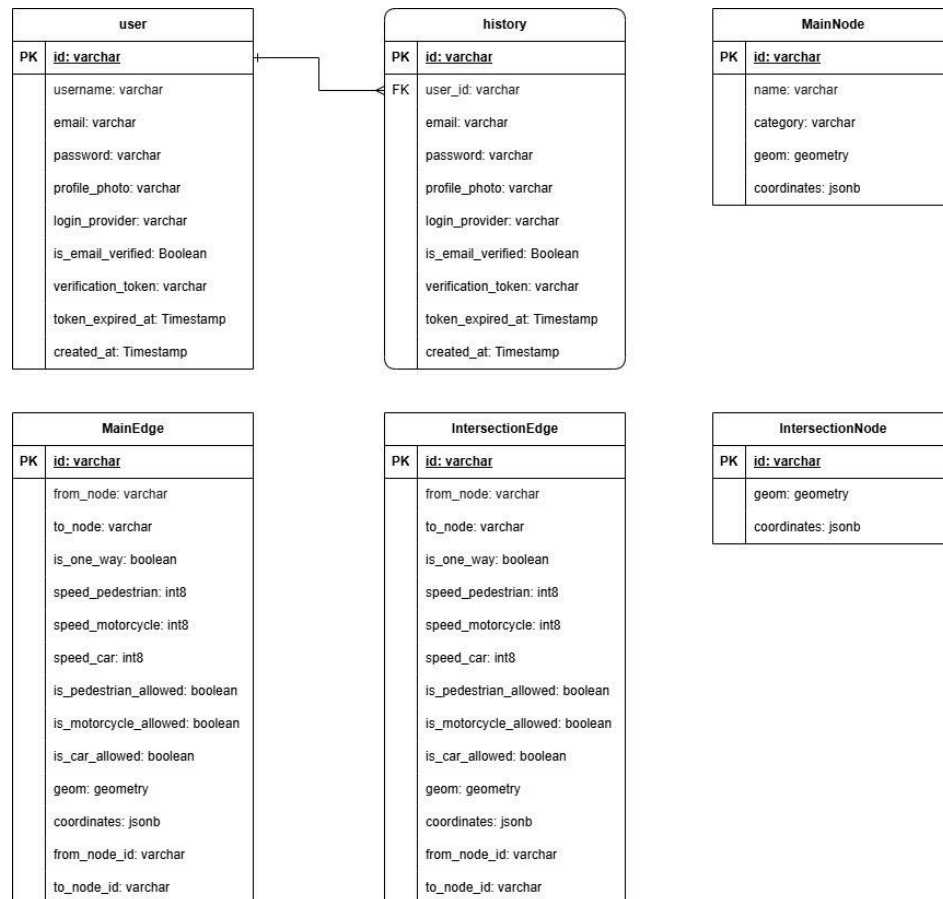
Spesifikasi perangkat yang direkomendasikan oleh penulis untuk mengembangkan, hingga menguji aplikasi secara local sebagai berikut agar proses pengembangan berjalan lancar tanpa kendala:

Processor : Intel Core i5 / AMD Ryzen 5 Generasi Terbaru
RAM : Minimal 8 GB
Storage : 256 GB (Wajib SSD)
Internet : Bandwith > 20 Mbps
Sistem Operasi : Windows 11/Disesuaikan Kembali dengan kenyamanan pengguna
IDE / Text Editor : Visual Studio Code Latest
Web Browser : Microsoft Edge (karena optimalisasi ramnya ketika minimize page)

3.2 Implementasi Basis Data

3.2.1 Entity Relationship Diagram (ERD)

Menampilkan dan menjelaskan diagram ERD



Gambar 1 ERD

Entitas Users menyimpan data autentikasi dan profil pengguna pada sistem. Entitas ini merupakan Strong Entity (Entitas Kuat) yang memiliki atribut id bertipe varchar sebagai Primary Key, username bertipe varchar, email bertipe varchar yang bersifat unique, password bertipe varchar yang bersifat nullable, profile_photo bertipe varchar yang bersifat nullable, login_provider bertipe varchar, is_email_verified bertipe boolean, verification_token bertipe varchar yang bersifat nullable, token_expired_at bertipe timestamp yang bersifat nullable, serta created_at bertipe timestamp yang mencatat waktu pembuatan data. Entitas ini berelasi dengan entitas History dengan kardinalitas One-to-Many (1:N), di mana satu pengguna dapat memiliki banyak riwayat pencarian, namun setiap riwayat pencarian hanya dimiliki oleh satu pengguna.

Entitas History menyimpan data riwayat rute yang pernah dicari oleh pengguna pada sistem. Entitas ini merupakan Weak Entity yang keberadaannya bergantung pada entitas Users. Entitas ini memiliki atribut id bertipe varchar sebagai Primary Key, user_id bertipe varchar sebagai Foreign Key yang merujuk ke users(id), start_point_name bertipe varchar yang menyimpan nama titik awal rute, end_point_name bertipe varchar yang menyimpan nama titik akhir rute, distance bertipe float8 yang menyimpan

jarak tempuh, duration bertipe float8 yang menyimpan estimasi waktu perjalanan, transport_mode bertipe varchar yang menyimpan moda transportasi yang digunakan, serta created_at bertipe timestamp yang mencatat waktu pencarian dilakukan. Entitas ini berelasi dengan entitas Users dengan kardinalitas Many-to-One (N:1), di mana banyak riwayat pencarian dapat dimiliki oleh satu pengguna, namun setiap riwayat pencarian hanya dapat dikaitkan dengan satu pengguna.

Entitas MainNode menyimpan data titik lokasi utama pada sistem, seperti gedung, asrama, dan fasilitas kampus. Entitas ini merupakan Strong Entity yang memiliki atribut id bertipe varchar sebagai Primary Key, name bertipe varchar yang menyimpan nama lokasi, category bertipe varchar yang menyimpan kategori lokasi, serta geom bertipe geometry dengan tipe point yang menyimpan koordinat geografis lokasi tersebut. Entitas ini berelasi secara logis dengan entitas Edge, di mana id dari entitas MainNode dihubungkan ke kolom from_node atau to_node pada entitas Edge untuk merepresentasikan titik awal dan titik akhir dari sebuah jalur. Relasi ini tidak menggunakan Foreign Key fisik, namun secara konseptual membentuk hubungan yang menggambarkan konektivitas antar lokasi dalam sistem navigasi.

Entitas IntersectionNode menyimpan data titik persimpangan jalan murni pada sistem tanpa memiliki nama lokasi tertentu. Entitas ini merupakan Strong Entity yang memiliki atribut id bertipe varchar sebagai Primary Key, serta geom bertipe geometry dengan tipe point yang menyimpan koordinat geografis titik persimpangan tersebut. Entitas ini berelasi secara logis dengan entitas Edge, di mana id dari entitas IntersectionNode dihubungkan ke kolom from_node atau to_node pada entitas Edge untuk merepresentasikan titik persimpangan sebagai bagian dari jalur navigasi. Sama seperti entitas MainNode, relasi ini tidak menggunakan Foreign Key fisik, namun secara konseptual berperan sebagai penghubung antar segmen jalan dalam membentuk jaringan navigasi pada sistem.

Entitas MainEdge menyimpan data segmen jalan utama yang dapat dilalui kendaraan pada sistem. Entitas ini merupakan Strong Entity yang memiliki atribut id bertipe varchar sebagai Primary Key, from_node dan to_node bertipe varchar sebagai referensi logis ke entitas Node yang merepresentasikan titik awal dan titik akhir segmen jalan, is_one_way bertipe boolean yang menandakan apakah jalan bersifat satu arah, speed_pedestrian, speed_motorcycle, dan speed_car bertipe int8 yang menyimpan kecepatan masing-masing moda transportasi, is_pedestrian_allowed, is_motorcycle_allowed, dan is_car_allowed bertipe boolean yang menandakan izin akses tiap moda transportasi, serta geom bertipe geometry linestring yang menyimpan representasi geografis segmen jalan. Entitas ini berelasi secara logis dengan entitas MainNode dan IntersectionNode, di mana setiap segmen menghubungkan dua node dalam graf spasial untuk membentuk jaringan jalan pada sistem navigasi.

Entitas IntersectionEdge menyimpan data segmen jalan pada area persimpangan atau area kecil dalam sistem. Entitas ini merupakan Strong Entity yang memiliki atribut yang sama persis dengan entitas MainEdge, yaitu id bertipe varchar sebagai Primary Key, from_node dan to_node bertipe varchar sebagai referensi logis ke entitas Node yang merepresentasikan titik awal dan titik akhir segmen jalan, is_one_way bertipe boolean yang menandakan apakah jalan bersifat satu arah, speed_pedestrian, speed_motorcycle, dan speed_car bertipe int8 yang menyimpan kecepatan masing-masing moda transportasi, is_pedestrian_allowed, is_motorcycle_allowed, dan is_car_allowed bertipe boolean yang menandakan izin akses tiap moda transportasi, serta geom bertipe geometry linestring yang menyimpan representasi geografis segmen jalan. Entitas ini berelasi secara logis dengan entitas MainNode dan IntersectionNode, di mana setiap segmen menghubungkan dua node dalam graf spasial untuk membentuk jaringan jalan pada sistem navigasi.

3.2.2 Struktur Basis data

3.2.2.1. Tabel User

Tabel *User* dirancang sebagai berikut;

Tabel 1 Tabel Entitas User

<i>Field</i>	<i>Tipe</i>	<i>Key</i>
ID	VARCHAR	<i>Primary Key</i>
USERNAME	VARCHAR	-
PASSWORD	VARCHAR	-
EMAIL	VARCHAR	-
PROFILE PHOTO	VARCHAR	-
LOGIN PROVIDER	VARCHAR	-
IS EMAIL VERYFIED	BOOLEAN	-
VERIFICATION TOKEN	VARCHAR	-
TOKEN EXPIRED AT	TIMESTAMP	-
CREATED AT	TIMESTAMP	-

3.2.2.2. Tabel History

Tabel 2 Tabel Entitas History

<i>Field</i>	<i>Tipe</i>	<i>Key</i>
ID	VARCHAR	<i>Primary Key</i>

USER ID	VARCHAR	<i>Foreign Key</i>
PASSWORD	VARCHAR(500)	-
EMAIL	VARCHAR	-
PROFILE PHOTO	VARCHAR	-
LOGIN PROVIDER	VARCHAR	-
IS EMAIL VERYFIED	BOOLEAN	-
VERIFICATION TOKEN	VARCHAR	-
TOKEN EXPIRED AT	TIMESTAMP	-
CREATED AT	TIMESTAMP	-

3.2.2.3. Tabel MainNode

Tabel 3 Tabel Entitas MainNode

<i>Field</i>	<i>Type</i>	<i>Key</i>
ID	VARCHAR	<i>Primary Key</i>
FROM_NODE	VARCHAR	-
TO_NODE	VARCHAR	-
IS_ONE_WAY	BOOLEAN	-
SPEED_PEDESTRIAN	INT(8)	-
SPEED_MOTORCYCLE	INT(8)	-
SPEED_CAR	INT(8)	-
IS_PEDESTRIAN_ALLOWED	BOOLEAN	-
IS_MOTORCYCLE_ALLOWED	BOOLEAN	-
IS_CAR_ALLOWED	BOOLEAN	-
GEOM	GEOMETRY	-
COORDINATES	JSONB	-

3.2.2.4. Tabel IntersectionNode

Tabel 4 Tabel Entitas IntersectionNode

<i>Field</i>	<i>Type</i>	<i>Key</i>
ID	VARCHAR	<i>Primary Key</i>
COORDINATES	JSONB	-

3.2.2.5. Tabel MainEdge

Tabel 5 Tabel Entitas MainEdge

<i>Field</i>	<i>Type</i>	<i>Key</i>
ID	VARCHAR	<i>Primary Key</i>
NAME	VARCHAR	-
GEOM	GEOMETRY	-
CATEGORY	VARCHAR	-
COORDINATES	JSONB	-

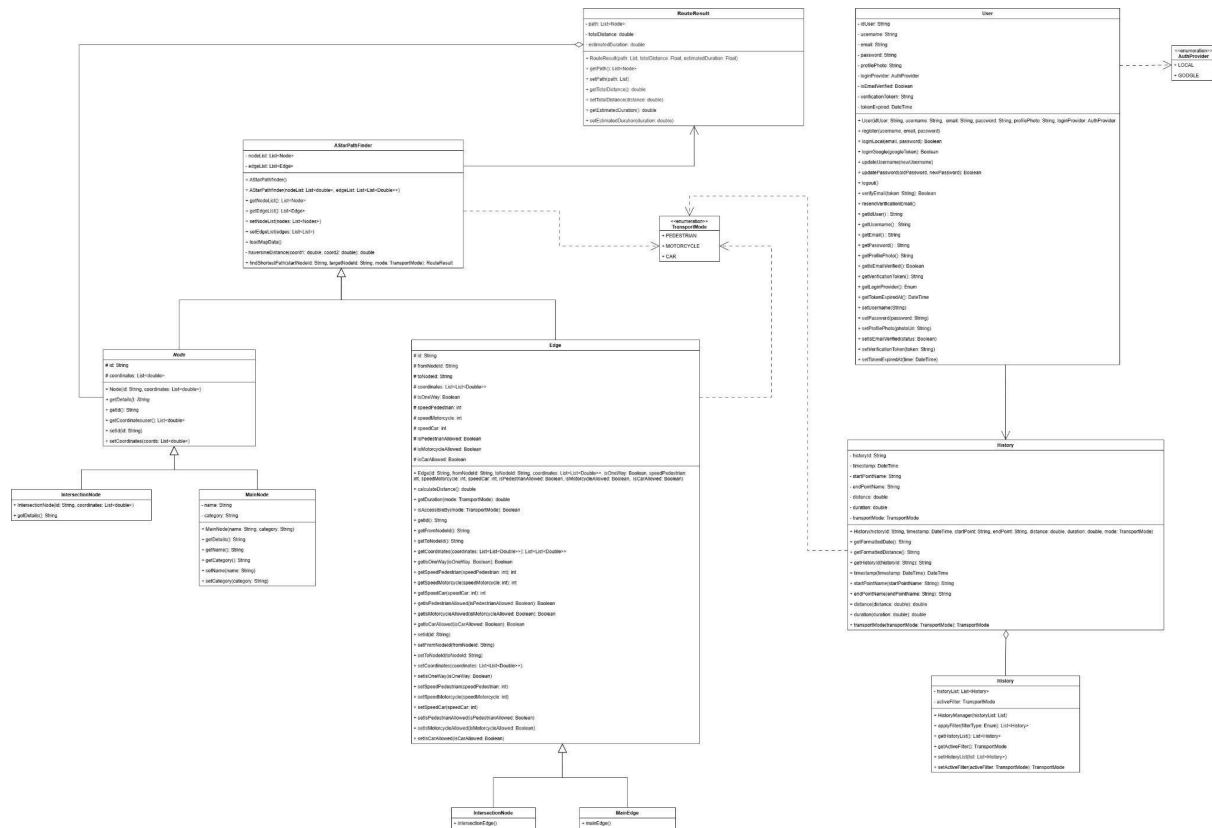
3.2.2.6. Tabel IntersectionEdge

Tabel 6 Tabel Entitas IntersectionEdge

<i>Field</i>	<i>Type</i>	<i>Key</i>
ID	VARCHAR	<i>Primary Key</i>
FROM_NODE	VARCHAR	-
TO_NODE	VARCHAR	-
IS_ONE_WAY	BOOLEAN	-
SPEED_PEDESTRIAN	INT(8)	-
SPEED_MOTORCYCLE	INT(8)	-
SPEED_CAR	INT(8)	-
IS_PEDESTRIAN_ALLOWED	BOOLEAN	-
IS_MOTORCYCLE_ALLOWED	BOOLEAN	-
IS_CAR_ALLOWED	BOOLEAN	-
GEOM	GEOMETRY	-
COORDINATES	JSONB	-

3.3.1 Class Diagram

Gambar 2 Class Diagram UML



3.3.2 Deskripsi Kelas

3.3.2.1. Kelas User

Kelas ini bertanggung jawab mengelola akun dan profil pengguna. Atribut yang dimiliki meliputi `userId`, `username`, `email`, `password` (dapat bernilai null jika pengguna login menggunakan Google), `profilePhoto` berupa URL foto, dan `loginProvider` berupa enum dengan nilai `LOCAL` atau `GOOGLE`. Selain itu, kelas ini memiliki atribut `isEmailVerified`, `verificationToken`, dan `tokenExpiredAt` untuk keperluan verifikasi email pengguna.

Kelas ini memiliki method `register()`, `loginLocal()`, `loginGoogle()`, `updateUsername()`, `updatePassword()`, dan `logout()`. Khusus untuk `updatePassword()`, terdapat pengecekan bahwa method ini tidak dapat dijalankan apabila `loginProvider` bernilai `GOOGLE`, karena kredensial pengguna Google dikelola sepenuhnya oleh penyedia OAuth melalui Supabase Auth.

Method `loginGoogle(googleToken)` merepresentasikan alur autentikasi menggunakan akun Google. Pada implementasi aktual, method ini mendelegasikan proses validasi token ke layanan Supabase Auth melalui mekanisme OAuth 2.0, yang kemudian mengembalikan JWT untuk sesi pengguna. Method ini tetap dimodelkan di class diagram untuk merepresentasikan tanggung jawab kelas User dalam menangani alur login Google secara konseptual.

Kelas User juga memiliki method `resendVerificationEmail()` yang berfungsi mengirim ulang email verifikasi apabila email sebelumnya tidak diterima atau telah kedaluwarsa. Method ini akan menghasilkan token baru yang disimpan pada atribut `verificationToken` dengan waktu kedaluwarsa baru pada `tokenExpiredAt`, kemudian dikirimkan kembali ke alamat email terdaftar.

Kelas User berelasi One-to-Many dengan kelas History, artinya satu pengguna dapat memiliki banyak riwayat perjalanan.

3.3.2.2. Kelas Node (Abstract)

Kelas abstrak yang menjadi induk dari `MainNode` dan `IntersectionNode`. Atributnya bersifat `protected`, yaitu `id` dan `coordinates`, agar dapat diakses langsung oleh kelas turunannya. Kelas ini dilengkapi constructor serta getter dan setter untuk kedua atributnya. Method `getDetails()` bersifat abstrak dan wajib diimplementasikan oleh setiap kelas turunan.

3.3.2.3. Kelas MainNode

Merupakan turunan dari kelas Node yang merepresentasikan lokasi utama di kampus seperti gedung atau fasilitas. Memiliki atribut tambahan bersifat `private`, yaitu `name` dan `category`, beserta getter dan setter masing-masing. Constructor kelas ini memanggil `super(id, coordinates)` dari kelas induk sebelum menginisialisasi atribut tambahannya. Method `getDetails()` mengimplementasikan method abstrak dari kelas induk dan mengembalikan nama serta kategori lokasi tersebut.

3.3.2.4. Kelas IntersectionNode

Merupakan turunan dari kelas Node yang merepresentasikan titik persimpangan jalan di peta kampus. Tidak memiliki atribut maupun method tambahan; seluruhnya diwarisi dari kelas Node. Constructor kelas ini hanya memanggil `super(id, coordinates)`, dan implementasi `getDetails()` mengembalikan informasi dasar berupa `id` node tersebut.

3.3.2.5. Kelas Edge (Abstract)

Kelas abstrak yang menjadi induk dari `MainEdge` dan `IntersectionEdge`, merepresentasikan segmen jalan yang

menghubungkan dua node. Atributnya bersifat protected, meliputi id, fromNodeId, toNodeId, coordinates berupa array of arrays (LineString), isOneWay, serta atribut kecepatan dan aksesibilitas untuk masing-masing moda transportasi (speedPedestrian, speedMotorcycle, speedCar, isPedestrianAllowed, isMotorcycleAllowed, isCarAllowed). Kelas ini dilengkapi getter dan setter untuk seluruh atributnya. Method yang dimiliki adalah calculateDistance() untuk menghitung panjang segmen jalan, getDuration(mode) untuk menghitung estimasi waktu tempuh berdasarkan moda, dan isAccessibleBy(mode) untuk mengecek apakah suatu moda diizinkan melewati edge tersebut.

3.3.2.6. Kelas MainEdge

Merupakan turunan dari kelas Edge yang merepresentasikan segmen jalan utama di kawasan kampus. Atribut dan method sepenuhnya diwarisi dari kelas Edge; constructor-nya memanggil super(...) dengan parameter yang sama seperti kelas induk.

3.3.2.7. Kelas IntersectionEdge

Merupakan turunan dari kelas Edge yang merepresentasikan segmen jalan di area persimpangan. Atribut dan method sepenuhnya diwarisi dari kelas Edge; constructor-nya memanggil super(...) dengan parameter yang sama seperti kelas induk.

3.3.2.8. Kelas AStarPathfinder

Merupakan kelas inti yang mengimplementasikan algoritma pencarian rute A*. Atributnya bersifat private, yaitu nodeList dan edgeList, masing-masing dilengkapi getter dan setter. Kelas ini menyediakan dua opsi constructor: constructor kosong (data diisi belakangan melalui setter atau method loadMapData()) dan constructor dengan parameter nodeList serta edgeList. Method loadMapData() digunakan untuk memuat data peta kampus dari sumber data GeoJSON.

Method haversineDistance(coord1, coord2) bersifat private karena hanya digunakan sebagai fungsi heuristik internal algoritma dan tidak dipanggil dari luar kelas. Method findShortestPath(startNodeId, targetNodeId, mode) bersifat publik dan menjalankan algoritma A* untuk mengembalikan objek RouteResult yang berisi path, total jarak, dan estimasi durasi perjalanan.

3.3.2.9. Kelas History

Kelas ini menyimpan riwayat perjalanan pengguna. Atributnya bersifat private, meliputi historyId, timestamp, startPointName,

endPointName, distance, duration, dan transportMode berupa enum. Kelas ini dilengkapi constructor untuk inisialisasi seluruh atribut serta getter untuk masing-masing atribut. Setter bersifat opsional karena data riwayat umumnya bersifat tetap (immutable) setelah dibuat, untuk menjaga validitas data. Method getFormattedDate() mengubah timestamp menjadi format yang mudah dibaca (contoh: "15 Juni 2026"), dan getFormattedDistance() mengonversi jarak dari meter ke kilometer. Kelas ini berelasi Many-to-One dengan kelas User.

3.3.2.10. Kelas HistoryManager

Kelas ini mengelola logika filter pada layar riwayat perjalanan. Atributnya bersifat private, yaitu historyList dan activeFilter berupa enum dengan nilai ALL, TODAY, YESTERDAY, LAST_WEEK, atau LAST_MONTH — nilai enum ini juga dicantumkan pada class diagram sebagai detail tipe data atribut activeFilter. Masing-masing atribut dilengkapi getter dan setter. Constructor kelas ini menerima parameter historyList. Method applyFilter(filterType) digunakan untuk menyaring daftar riwayat sesuai kategori waktu yang dipilih pengguna.

3.3.2.11. Kelas RouteResult

Kelas ini menyimpan hasil pencarian rute yang dikembalikan oleh AStarPathfinder. Atributnya bersifat private, meliputi path berupa list Node yang merepresentasikan urutan node pada rute yang ditemukan, totalDistance berupa float yang menyimpan total jarak tempuh dalam meter, dan estimatedDuration berupa float yang menyimpan estimasi durasi perjalanan. Kelas ini dilengkapi constructor untuk inisialisasi seluruh atribut serta getter dan setter untuk masing-masing atribut. Kelas RouteResult berelasi agregasi dengan kelas Node, dan merupakan objek yang berasosiasi dengan kelas AStarPathfinder.

3.4 Implementasi Antarmuka Pengguna (*User Interface*)

3.4.1 Halaman *Login*

Halaman ini merupakan pintu masuk pengguna ke dalam aplikasi. Pengguna dapat masuk menggunakan email dan password yang telah terdaftar, atau melalui opsi *login* cepat menggunakan akun Google. Pada halaman ini juga tersedia tautan "Forgot Password" untuk pengguna yang lupa kata sandi, serta tautan untuk berpindah ke halaman registrasi bagi pengguna yang belum memiliki akun.

3.4.2 Halaman Registrasi

Halaman ini digunakan oleh pengguna baru untuk mendaftarkan akun ke dalam sistem. Pengguna mengisi data berupa *username*, email, dan password pada formulir yang disediakan, kemudian menekan tombol "Create Account" untuk menyelesaikan proses registrasi. Halaman ini juga menyediakan tautan untuk kembali ke halaman *login* bagi pengguna yang sudah memiliki akun.

3.4.3 Halaman Forgot Password

Halaman ini terdiri dari dua tahap. Pada tahap pertama, pengguna memasukkan alamat email terdaftar untuk menerima kode OTP, lalu memasukkan kode tersebut untuk verifikasi identitas. Setelah verifikasi berhasil, pengguna diarahkan ke tahap kedua untuk membuat password baru dan mengonfirmasinya sebelum menekan tombol "Change Password".

3.4.4 Halaman Utama (*Main Page*)

Halaman ini merupakan layar utama aplikasi yang menampilkan peta kawasan kampus Telkom University secara *full-screen*. Pengguna dapat memilih moda transportasi (Walk, Motor, atau Car) melalui tombol pilihan yang tersedia di bagian atas panel informasi. Panel ini juga menampilkan titik asal dan tujuan yang telah dipilih, beserta ringkasan estimasi waktu tempuh dan jarak. Rute yang ditemukan ditampilkan sebagai garis (*polyline*) di atas peta, dan pengguna dapat menekan tombol "Start" untuk memulai navigasi.

3.4.5 Halaman Pengaturan (*Setting Page*)

Halaman ini menampilkan informasi profil pengguna beserta tombol *logout*. Pada bagian *Account Settings*, pengguna dapat mengubah *username*, password, dan email. Pada bagian *System Settings*, pengguna dapat mengaktifkan mode tampilan gelap (*Dark Mode*) dan mengubah bahasa aplikasi (*Language*).

4 Pengujian Sistem

Bagian ini membahas proses pengujian yang dilakukan terhadap sistem yang telah dibangun, meliputi tujuan pengujian, lingkungan pengujian, metode pengujian yang digunakan, serta skenario pengujian pada setiap fitur sistem.

4.1 Tujuan Pengujian

Pengujian sistem dilakukan untuk memastikan bahwa seluruh fitur yang telah dirancang dan diimplementasikan berfungsi sesuai dengan kebutuhan yang telah diidentifikasi pada bagian 3.1.1. Pengujian ini bertujuan untuk menemukan kesalahan (*bug*) pada sistem, memverifikasi bahwa setiap fitur memberikan keluaran yang sesuai dengan masukan yang diberikan, serta memastikan sistem dapat menangani kondisi tidak normal (misalnya input kosong atau salah) dengan baik sebelum sistem digunakan oleh pengguna akhir.

4.2 Lingkungan Pengujian

Pengujian sistem dilakukan pada lingkungan perangkat lunak dan perangkat keras tertentu untuk memastikan hasil pengujian konsisten dan dapat dipertanggungjawabkan. Berikut adalah spesifikasi lingkungan yang digunakan selama proses pengujian.

4.2.1 Perangkat Lunak Pengujian

Sistem ini diujikan dengan spesifikasi perangkat lunak sebagai berikut;

- Sistem Operasi: Windows 11
- Bahasa Pemrograman: Java (backend), TypeScript (frontend)
- *Framework* Pengujian Backend: Spring Boot Test dengan H2 *Database (in-memory)*
- *Framework* Pengujian Frontend: Jest dan Testing Library React Native
- Aplikasi Pengujian Mobile: Expo Go
- *Web Browser*: Microsoft Edge

4.2.2 Perangkat Keras Pengujian

Sistem ini diujikan dengan spesifikasi perangkat Keras sebagai berikut;

- Prosesor: Intel Core i5 / AMD Ryzen 5 generasi terbaru
- Memori (RAM): Minimal 8 GB
- *Storage*: 256 GB SSD
- Perangkat Mobile: Smartphone Android/iOS untuk pengujian melalui Expo Go

4.3 Metode Pengujian

Pengujian sistem dilakukan menggunakan metode tertentu untuk memvalidasi bahwa setiap fitur berjalan sesuai dengan kebutuhan yang telah dirancang. Berikut adalah metode pengujian beserta skenario yang digunakan pada sistem ini.

4.3.1 (Black Box/White Box) Testing

Pengujian sistem dilakukan menggunakan metode *Black Box Testing*, yaitu metode pengujian yang berfokus pada keluaran sistem berdasarkan masukan yang diberikan, tanpa memeriksa struktur kode atau alur logika internal program. Metode ini dipilih karena bertujuan untuk memvalidasi bahwa setiap fitur berjalan sesuai spesifikasi fungsional dari sudut pandang pengguna.

4.3.2 Skenario Pengujian

Pengujian dilakukan pada seluruh fitur utama sistem. Skenario pengujian dapat dilihat sebagai berikut

Tabel 7 Identifikasi Pengujian

Kelas Uji	Butir Uji	Identifikasi	Jenis Pengujian	Penguji
<i>Autentikasi</i>	Pengujian fungsi login lokal	AO-AUTH1	Black Box	Hamam
	Pengujian fungsi login Google	AO-AUTH2	Black Box	Hamam
	Pengujian validasi login gagal	AO-AUTH3	Black Box	Hamam
Pemilihan Titik Asal & Tujuan	Pengujian pemilihan titik di peta	AO-POINT 01	Black Box	Najla
Pemilihan Moda Transportasi	Pengujian pemilihan moda WALK/MOT ORCYCLE/CAR	AO-MODE 01	Black Box	Putri Rahayu
Pencarian Rute Terpendek	Pengujian pencarian rute berhasil	AO-ROUT E01	Black Box	Najla
	Pengujian pencarian rute tanpa jalur tersedia	AO-ROUT E02	Black Box	Najla
Kalkulasi Jarak & Estimasi Waktu	Pengujian akurasi jarak dan durasi	AO-CALC0 1	Black Box	Putri Ayu
Tampilan Hasil Rute	Pengujian tampilan polyline dan ringkasan rute	AO-DISPL AY01	Black Box	Putri Rahayu
	Pengujian hapus rute aktif	AO-DISPL AY02	Black Box	Putri Rahayu
Riwayat Perjalanan	Pengujian penyimpanan riwayat	AO-HIST01	Black Box	Putri Ayu
	Pengujian filter riwayat	AO-HIST02	Black Box	Putri Ayu

	berdasarkan waktu			
Pengaturan Akun & Tema	Pengujian ubah username/pass word/foto profil	AO-SET01	Black Box	Hamam
	Pengujian ubah tema light/dark mode	AO-SET02	Black Box	Hamam

5 Hasil Uji

Hasil uji berdasarkan skenario yang telah dibuat pada bab sebelumnya.

5.1 Fitur Autentikasi

5.1.1 Kasus uji

Beberapa kasus uji yang dilakukan sebagai berikut;

Tabel 8 Kasus Uji Fitur Autentikasi

Identifikasi	Skenario
AO-AUTH1	Login lokal berhasil dengan email dan password yang benar
AO-AUTH2	Login menggunakan akun Google berhasil
AO-AUTH3	Login lokal gagal karena email tidak terdaftar
AO-AUTH3	Login lokal gagal karena password salah
AO-AUTH3	Login lokal gagal karena field email/password kosong

5.1.2 Hasil

Hasil Uji Sebagai Berikut

Tabel 9 Hasil Uji Fitur Autentikasi

Identifikasi	Input	Hasil yang diharapkan	Hasil sebenarnya	Status
AO-AUTH1	Email: saya@telkomuniversity.ac.id,	Login berhasil, masuk ke Main Page	Sesuai	Pengujian berhasil

	Password: Saya123			
AO-AUTH2	Akun Google terdaftar	Login berhasil melalui OAuth, masuk ke Main Page	Sesuai	Pengujian berhasil
AO-AUTH3	Email: tidak_ada@g mail.com, Password: Saya123	Pesan error "Email tidak ditemukan"	Sesuai	Pengujian berhasil
AO-AUTH3	Email: saya@telko muniversity. ac.id, Password: salah	Pesan error "Password salah"	Sesuai	Pengujian berhasil
AO-AUTH3	Email/Passw ord kosong	Validasi muncul, tombol Login tidak aktif	Sesuai	Pengujian berhasil

5.2 Fitur Pemilihan Titik Asal dan Tujuan

5.2.1 Kasus uji

Beberapa kasus uji yang dilakukan sebagai berikut;

Tabel 10 Hasil Uji Fitur Pemilihan Titik Asal dan Tujuan

Identifikasi	Skenario
AO-POINT01	Memilih titik asal dengan mengetuk peta
AO-POINT01	Memilih titik tujuan dengan mengetuk peta
AO-POINT01	Mencari rute tanpa memilih titik asal atau tujuan

5.2.2 Hasil

Hasil Uji Sebagai Berikut

Tabel 11 Hasil Uji Fitur Pemilihan Titik Asal dan Tujuan

Identifikasi	Input	Hasil yang diharapkan	Hasil sebenarnya	Status
AO-POINT01	Ketukan pada	Titik asal tersimpan dan	Sesuai	Pengujian berhasil

	koordinat gedung FIF	ditandai di peta		
AO-POINT01	Ketukan pada koordinat Gedung Damar	Titik tujuan tersimpan dan ditandai di peta	Sesuai	Pengujian berhasil
AO-POINT01	Tombol "Cari Rute" ditekan tanpa titik asal/tujuan terisi	Tombol tidak aktif / pesan peringatan muncul	Sesuai	Pengujian berhasil

5.3 Fitur Pemilihan Moda Transportasi

5.3.1 Kasus Uji

Tabel 12 Hasil Uji Fitur Pemilihan Moda Transportasi

Identifikasi	Skenario
AO-MODE01	Memilih moda transportasi jalan kaki (<i>WALK</i>) valid.
AO-MODE01	Memilih moda transportasi motor (<i>MOTORCYCLE</i>) valid.
AO-MODE01	Memilih moda transportasi mobil (<i>CAR</i>) valid.
AO-MODE01	Pengujian Aksesibilitas Jalan Non-Kendaraan (Jalur Khusus Pejalan Kaki/Gang Sempit).

5.3.2 Hasil

Tabel 13 Hasil Uji Fitur Pemilihan Moda Transportasi

Identifikasi	Input	Hasil yang diharapkan	Hasil sebenarnya	Status
AO-MODE01	Lokasi awal: TULT Lokasi Tujuan: Danau Galau Moda: Walk	Sistem berhasil menampilkan rute <i>walkway</i> dan estimasi waktu perjalanan.	Sesuai	Pengujian Berhasil

AO-MODE01	Lokasi awal: TULT Lokasi Tujuan: BTP Moda: Motor	Sistem berhasil menampilkan rute <i>walkway</i> dan estimasi waktu perjalanan.	Sesuai	Pengujian Berhasil
AO-MODE01	Lokasi awal: TULT Lokasi Tujuan: BTP Moda: Mobil	Sistem berhasil menampilkan rute <i>walkway</i> dan estimasi waktu perjalanan.	Sesuai	Pengujian Berhasil
AO-MODE01	Lokasi awal: Rabbit Park Lokasi Tujuan: BTP Moda: Motor	sistem menampilkan pesan error bahwa jalan tersebut tidak bisa dilewati motor/mobil	Sesuai	Pengujian Berhasil

5.4 Fitur Pencarian Rute Terpendek

5.4.1 Kasus uji

Tabel 14 Hasil Uji Fitur Pencarian Rute Terpendek

Identifikasi	Skenario
AO-ROUTE01	Pencarian rute antara Telkom University Landmark Tower ke Graha Wiyata Cacuk A
AO-ROUTE02	Pencarian rute pada moda CAR menuju area yang hanya bisa diakses pejalan kaki

5.4.2 Hasil

Tabel 15 Hasil Uji Fitur Pencarian Rute Terpendek

Identifikasi	Input	Hasil yang diharapkan	Hasil sebenarnya	Status
--------------	-------	-----------------------	------------------	--------

AO-ROUTE01	Titik asal: Telkom University Landmark Tower, Titik tujuan: Graha Wiyata Cacuk A Moda: CAR	Rute ditemukan, ditampilkan sebagai polyline di peta	Sesuai	Pengujian berhasil
AO-ROUTE02	Titik asal: Telkom University Landmark Tower, Titik tujuan: Telkom Rabit Park Moda: CAR	Rute ditemukan, ditampilkan sebagai polyline di peta	Error: No Path Found	Pengujian gagal

5.5 Fitur Kalkulasi Jarak dan Estimasi Waktu

5.5.1 Kasus uji

Tabel 16 Hasil Uji Fitur Kalkulasi Jarak dan Estimasi Waktu

Identifikasi	Skenario
AO-CALC01	Menghitung jarak dan durasi untuk moda WALK
AO-CALC02	Menghitung jarak dan durasi untuk moda MOTORCYCLE
AO-CALC03	Menghitung jarak dan durasi untuk moda CAR

5.5.2 Hasil

Tabel 17 Hasil Uji Fitur Kalkulasi Jarak dan Estimasi Waktu

Identifikasi	Input	Hasil yang diharapkan	Hasil sebenarnya	Status
AO-CALC01	Titik asal: Telkom University	Sistem menampilkan	Sistem berhasil menampilkan	Pengujian berhasil

	Landmark Tower, Titik tujuan: Telkom Rabbit Park Moda: WALK	hasil estimasi waktu.	estimasi waktu yaitu 10 menit	
AO-CALC02	Titik asal: Telkom University Landmark Tower, Titik tujuan: Asrama Putra Gedung 11 Park Moda: MOTORCYCLE	Sistem menampilkan hasil estimasi waktu.	Sistem berhasil menampilkan estimasi waktu yaitu 5 menit	Pengujian berhasil
AO-CALC03	Titik asal: Telkom University Landmark Tower, Titik tujuan: Asrama Putra Gedung 11 Park Moda: CAR	Sistem menampilkan hasil estimasi waktu.	Sistem berhasil menampilkan estimasi waktu yaitu 9 menit	Pengujian berhasil

5.6 Fitur Tampilan Hasil Rute

5.6.1 Kasus uji

Tabel 18 Hasil Uji Fitur Tampilan Hasil Rute

Identifikasi	Skenario
AO-DISPLAY01	Menampilkan polyline dan ringkasan rute setelah pencarian berhasil
AO-DISPLAY02	Menampilkan polyline dan ringkasan rute setelah pencarian berhasil

5.6.2 Hasil

Tabel 19 Hasil Uji Fitur Tampilan Hasil Rute

Identifikasi	Input	Hasil yang diharapkan	Hasil sebenarnya	Status
AO-DISPLA Y01	Rute ditemukan dari Telkom Rabbit Park ke Bandung Techno Park, moda: MOTORCYCLE	Polyline tampil di peta, ringkasan jarak dan estimasi waktu sampai tampil di panel	Polyline tampil di peta, estimasi jarak = 1.3 km dan estimasi waktu sampai = 5 min	Pengujian berhasil
AO-DISPLA Y02	Rute ditemukan dari Telkom University Landmark Tower ke Telkom Rabbit Park, moda: WALK	Polyline tampil di peta, ringkasan jarak dan estimasi waktu sampai tampil di panel	Polyline tampil di peta, estimasi jarak = 828 m dan estimasi waktu sampai = 10 min	Pengujian berhasil

5.7 Fitur Riwayat Perjalanan

5.7.1 Kasus uji

Tabel 20 Hasil Uji Fitur Riwayat Perjalanan

Identifikasi	Skenario
AO-HIST01	Penyimpanan riwayat setelah pencarian rute berhasil
AO-HIST01	Filter riwayat berdasarkan kategori waktu (Hari ini, Kemarin, Minggu lalu, Bulan lalu)

5.7.2 Hasil

Tabel 21 Hasil Uji Fitur Riwayat Perjalanan

Identifikasi	Input	Hasil yang diharapkan	Hasil sebenarnya	Status
AO-HIST01	Pencarian rute dari Gedung FIF ke Gedung Damar selesai	Riwayat baru tersimpan dengan data titik, jarak, durasi, dan moda	Sesuai	Pengujian berhasil
AO-HIST01	Filter "Hari ini" dipilih pada layar riwayat	Daftar riwayat hanya menampilkan pencarian pada hari yang sama	Sesuai	Pengujian berhasil

5.8 Fitur Pengaturan Akun dan Tema

5.8.1 Kasus uji

Tabel 22 Hasil Uji Fitur Pengaturan Akun dan Tema

Identifikasi	Skenario
AO-SET01	Mengubah username, password, dan foto profil
AO-SET01	Mencoba mengubah password pada akun login Google
AO-SET02	Mengubah tema dari light mode ke dark mode

5.8.2 Hasil

Tabel 23 Hasil Uji Fitur Pengaturan Akun dan Tema

Identifikasi	Input	Hasil yang diharapkan	Hasil sebenarnya	Status
AO-SET01	Username baru: "saya_s", Password baru: "SayaBaru 123"	Data tersimpan, profil diperbarui	Sesuai	Pengujian berhasil

AO-SET01	Akun dengan loginProvider GOOGLE mencoba ubah password	Pesan error "Tidak dapat mengubah password untuk akun Google"	Sesuai	Pengujian berhasil
AO-SET02	Tombol "Dark Mode" diaktifkan	Tampilan aplikasi berubah ke tema gelap	Sesuai	Pengujian berhasil

6 Penutup

Bab ini berisi kesimpulan yang diperoleh berdasarkan hasil perancangan, implementasi, dan pengujian sistem yang telah dilakukan. Selain itu, bab ini juga memuat saran-saran yang dapat dijadikan sebagai bahan pertimbangan untuk pengembangan dan penyempurnaan sistem pada masa yang akan datang agar dapat memberikan manfaat yang lebih optimal bagi pengguna.

6.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan, dapat disimpulkan bahwa aplikasi pencarian rute tercepat berbasis peta Telkom University berhasil dikembangkan sesuai dengan kebutuhan yang telah ditetapkan. Sistem mampu membantu pengguna dalam menentukan jalur tercepat menuju lokasi yang diinginkan di lingkungan kampus dengan memanfaatkan algoritma A* (A-Star Search) dan heuristik haversine untuk memperoleh rute yang optimal.

Aplikasi yang dibangun telah menyediakan berbagai fitur utama, seperti autentikasi pengguna, pemilihan titik asal dan tujuan, pemilihan moda transportasi, pencarian rute tercepat, perhitungan jarak dan estimasi waktu tempuh, penyimpanan riwayat perjalanan, serta pengaturan akun dan tema aplikasi. Berdasarkan hasil pengujian Black Box Testing, seluruh fitur yang diuji menunjukkan hasil yang sesuai dengan spesifikasi dan kebutuhan fungsional yang telah dirancang sebelumnya.

Selain itu, penggunaan teknologi Spring Boot pada sisi backend, React Native pada sisi frontend, serta PostgreSQL sebagai basis data berhasil mendukung proses pengembangan aplikasi yang terstruktur, responsif, dan dapat berjalan pada platform Android maupun iOS. Dengan demikian, tujuan pembangunan sistem untuk membantu mobilitas pengguna di lingkungan Telkom University dapat tercapai dengan baik.

6.2 Saran

Meskipun sistem telah berhasil diimplementasikan dan berjalan sesuai kebutuhan, masih terdapat beberapa peluang pengembangan yang dapat dilakukan pada penelitian atau pengembangan selanjutnya, antara lain:

1. Menambahkan fitur navigasi secara real-time dengan memanfaatkan GPS sehingga pengguna dapat memperoleh petunjuk arah secara langsung selama perjalanan.
2. Mengintegrasikan data kondisi lalu lintas atau kepadatan area kampus untuk menghasilkan rekomendasi rute yang lebih adaptif.
3. Menambahkan fitur pencarian lokasi berdasarkan nama gedung atau fasilitas tanpa harus memilih titik secara manual pada peta.
4. Memperluas cakupan peta agar tidak hanya mencakup area Telkom University, tetapi juga area di sekitar kampus.
5. Menambahkan fitur favorit atau bookmark lokasi yang sering dikunjungi pengguna untuk meningkatkan kemudahan penggunaan aplikasi.
6. Melakukan optimasi performa algoritma dan pengelolaan data peta agar sistem tetap responsif ketika jumlah node dan edge pada graf meningkat.

Dengan adanya pengembangan tersebut, diharapkan aplikasi dapat memberikan pengalaman navigasi yang lebih baik, akurat, dan bermanfaat bagi seluruh pengguna di lingkungan Telkom University.

Daftar Pustaka

IBM. (N.D.). ENUMERATIONS. IBM DOCUMENTATION. DIAKSES PADA 16 JUNI 2026, DARI [HTTPS://WWW.IBM.COM/DOCS/EN/DMA?TOPIC=DIAGRAMS-ENUMERATIONS](https://www.ibm.com/docs/en/dma?topic=diagrams-enumerations)

VISUAL PARADIGM. (N.D.). DRAWING CLASS DIAGRAMS. VISUAL PARADIGM USER'S GUIDE. DIAKSES PADA 16 JUNI 2026, DARI [HTTPS://WWW.VISUAL-PARADIGM.COM/SUPPORT/DOCUMENTS/VPUSERGUIDE/94/2576/7190_DRAWINGCLASS.HTML](https://www.visual-paradigm.com/support/documents/vpuserguide/94/2576/7190_DRAWINGCLASS.HTML)

ACCOUNTING DEPARTMENT BINUS UNIVERSITY. (2025, 18 JUNI). *ERD DAN CLASS DIAGRAM: APA BEDANYA DALAM PEMBUATAN DATABASE?* BINUS UNIVERSITY. DIAKSES PADA 16 JUNI 2026, DARI [HTTPS://ACCOUNTING.BINUS.AC.ID/2025/06/18/ERD-DAN-CLASS-DIAGRAM-APA-BEDANYA-DALAM-PEMBUATAN-DATABASE/](https://accounting.binus.ac.id/2025/06/18/erd-dan-class-diagram-apa-bedanya-dalam-pembuatan-database/)

FAKULTAS INFORMATIKA TELKOM UNIVERSITY. (2026). CLASS DIAGRAM. SCHOOL OF COMPUTING, TELKOM UNIVERSITY. FAKULTAS INFORMATIKA TELKOM UNIVERSITY. (2026). DARI [HTTPS://LMS.TELKOMUNIVERSITY.AC.ID/PLUGINFILE.PHP/8788867/MOD_RESOURCE/CONTENT/3/03%20-%20CLASS%20DIAGRAM.PDF](https://lms.telkomuniversity.ac.id/pluginfile.php/8788867/mod_resource/content/3/03%20-%20CLASS%20DIAGRAM.PDF)

FAKULTAS INFORMATIKA TELKOM UNIVERSITY. (2026). *INHERITANCE*. SCHOOL OF COMPUTING, TELKOM UNIVERSITY. DARI [HTTPS://LMS.TELKOMUNIVERSITY.AC.ID/PLUGINFILE.PHP/8788872/MOD_RESOURCE/CONTENT/3/04%20-%20INHERITANCE.PDF](https://lms.telkomuniversity.ac.id/pluginfile.php/8788872/mod_resource/content/3/04%20-%20INHERITANCE.PDF)

FAKULTAS INFORMATIKA TELKOM UNIVERSITY. (2026). *ABSTRACT CLASS AND INTERFACE* [SLIDE KULIAH]. SCHOOL OF COMPUTING, TELKOM UNIVERSITY. DARI [HTTPS://LMS.TELKOMUNIVERSITY.AC.ID/PLUGINFILE.PHP/8788877/MOD_RESOURCE/CONTENT/3/05%20-%20ABSTRACT%20CLASS%20AND%20INTERFACE.PDF](https://lms.telkomuniversity.ac.id/pluginfile.php/8788877/mod_resource/content/3/05%20-%20ABSTRACT%20CLASS%20AND%20INTERFACE.PDF)

Lampiran

Lampiran A Pembagian Tugas

NIM	Nama	Tugas	Kontribusi (100%)	Penyelesaian (%)
13012400305	Najla Tsabita Afiyah	1. 2. 3.		100%
13012430055	Putri Ayu Lestari	1.		100%
13012400118	Hamam Nashiruddin	1. 2.		100%
13012400277	Putri Rahayu Damayanti	1.		100%

Lampiran B Source Code Penting

Link Github: https://github.com/hmmntz20/TubesPBO_Kelompok-3_IF4806

Lampiran C Screenshot Sistem

Gambar 3&4



Welcome Back

Sign in to continue

Email

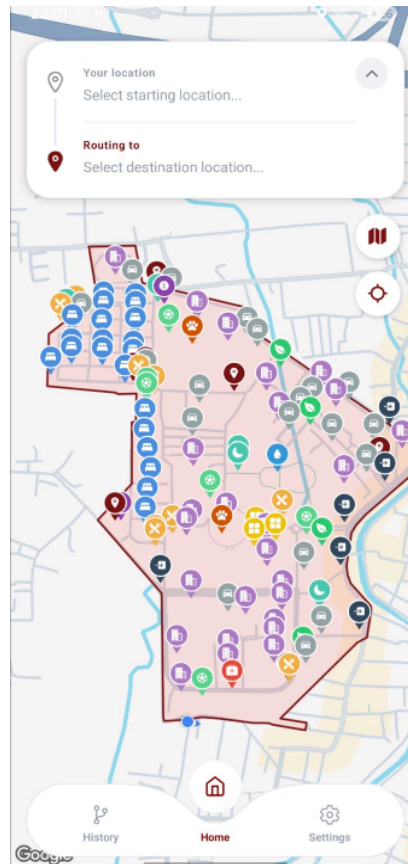
Password

Sign In

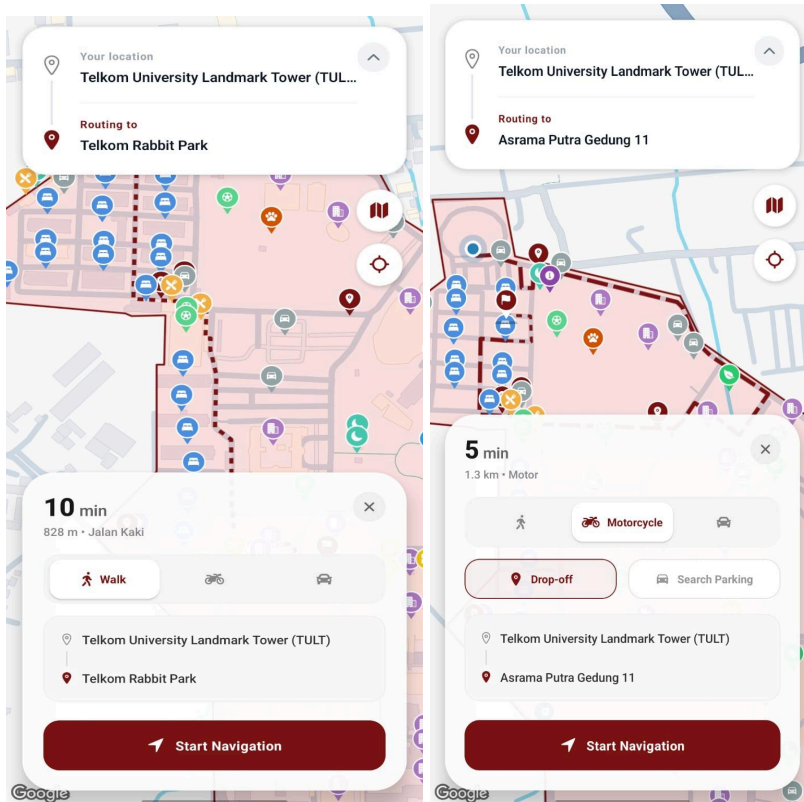
OR

 Continue with Google

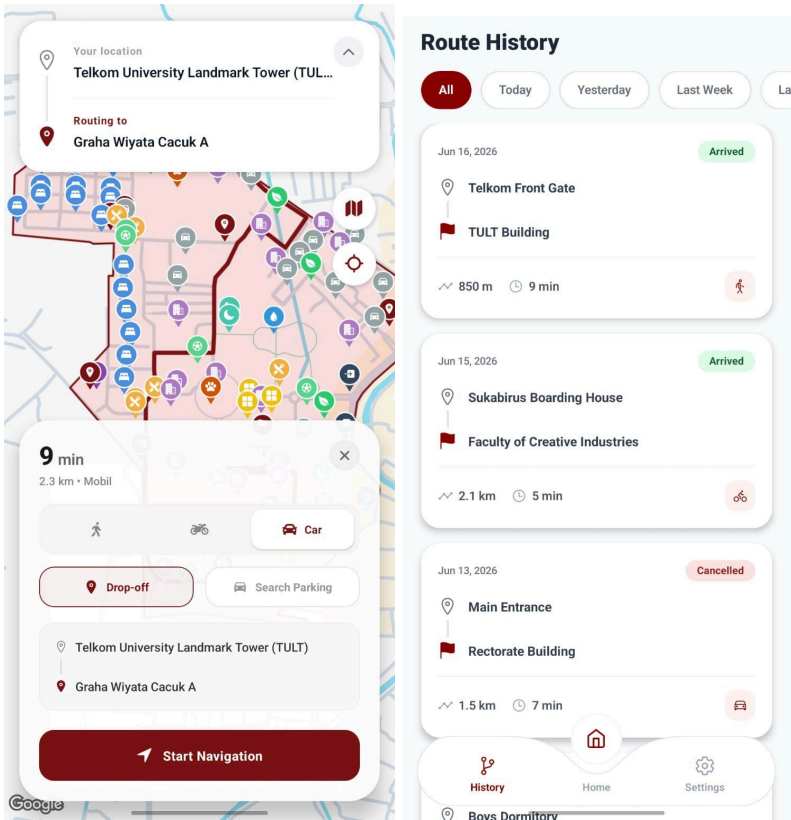
Don't have an account? [Sign Up](#)



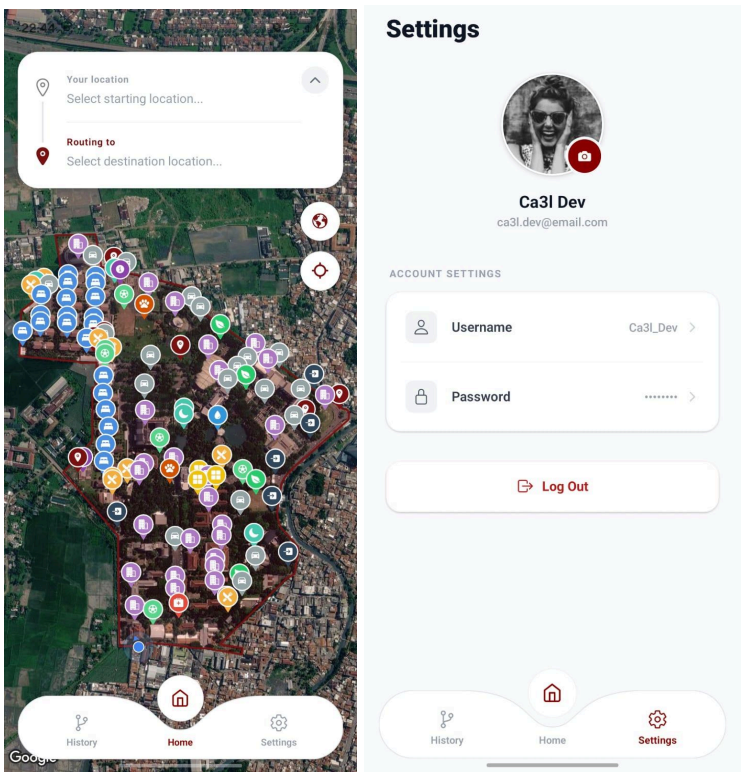
Gambar 5&6



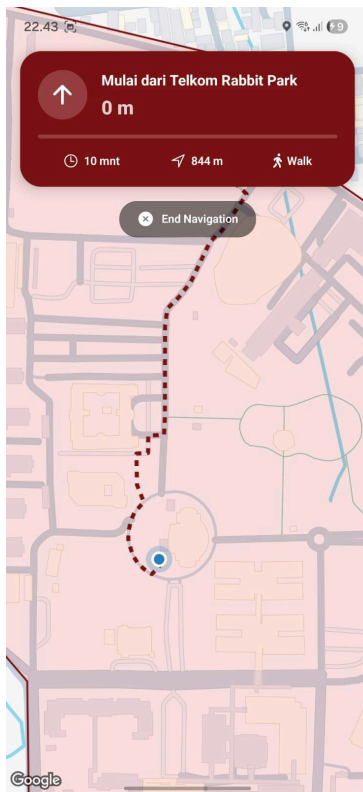
Gambar 7&8



Gambar 9&10



Gambar 11



Lampiran D Manual Instalasi Sistem

1. Buka GitHub projek melalui URL ini
https://github.com/hmmntz20/TubesPBO_Kelompok-3_IF4806
2. Lakukan Clone Projek dengan lewat Command Prompt dengan perintah: `git clone https://github.com/hmmntz20/TubesPBO_Kelompok-3_IF4806.git`
3. Buka Projek pada visual studio code (atau source code editor lainnya), lalu pada terminal ketik `npm i` untuk menginstal semua package/dependency

```

PS D:\pboTubes\TubesPBO\TubesPBO> npm i

up to date, audited 1030 packages in 7s

202 packages are looking for funding
  run `npm fund` for details

21 moderate severity vulnerabilities

To address issues that do not require attention, run:
  npm audit fix

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
PS D:\pboTubes\TubesPBO\TubesPBO>

```

4. Selanjutnya ketik `npx expo start` jika belum pernah membuka aplikasi, jika sudah pernah maka ketik `npx expo start -clear`

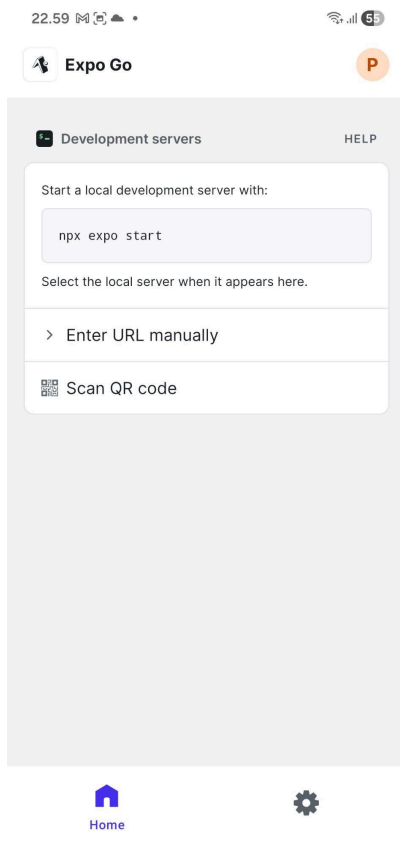
```

PS D:\pboTubes\TubesPBO\TubesPBO> npx expo start --clear
Starting project at D:\pboTubes\TubesPBO\TubesPBO
React Compiler enabled
Starting Metro Bundler
warning: Bundler cache is empty, rebuilding (this may take a minute)
The following packages should be updated for best compatibility with the installed expo version:
  react-native-svg@15.15.5 - expected version: 15.12.1
  babel-preset-expo@56.0.15 - expected version: ~54.0.10
Your project may not work correctly until you install the expected versions of the packages.

```



5. Setelah di enter akan ada barcode yang diberikan dan selanjutnya scan QR tersebut dengan menggunakan aplikasi Expo Go.



6. Ketika sudah di scan akan ada tampilan seperti ini ketik J lalu enter pada Proceed Anonymously.

```
> Using Expo Go
> Press s | switch to development build

> Press a | open Android
> Press w | open web

> Press j | open debugger
> Press r | reload app
> Press m | toggle menu
> shift+m | more tools
> Press o | open project code in your editor

> Press ? | show all commands

Logs for your project will appear below. Press Ctrl+C to exit.
?

It is recommended to log in with your Expo account before proceeding.
Learn more: https://expo.fyi/unverified-app-expo-go
» - Use arrow-keys. Return to submit.
> Log in
  Proceed anonymously
```

7. Selanjutnya akan ada loading sebentar jika sudah 100% maka aplikasi siap digunakan



Welcome Back

Sign in to continue

Email

your@email.com

Password



Sign In

OR



Continue with Google

Don't have an account? [Sign Up](#)